

ABRA — The Special Cut

The whole story, front to back: the cast, how they interact, the win-probability ceiling, and every white paper.

Will Hooper · 2026-07-23

A single consecutive read of everything ABRA. Start at the cheat card, glance at the interaction chart, read the predictability study for the CHOMP/JOLTEON/win% dynamic, then the white papers. Updated in the same pass as the code.

Contents

1. The cast, on one card
2. How the models interact
3. Executive Summary
4. The Deck (plain English)
5. Predictability Study — CHOMP, JOLTEON & the win% ceiling
6. White Paper — the data platform
7. White Paper — learning the simulator (the ML)
8. White Paper — SLOWKING (offline belief-state search)
9. SLOWKING — research roadmap
10. KADABRA — the coach spec
11. Architecture notes (languages, server, recency)

ABRA — the cast, on one card

Version 1.0 · 2026-07-22 · Will Hooper

Every model is a Pokémon, and every name is an acronym. For the **three battle-playing tiers only** — JOLTEON → MEDICHAM → SLOWKING — the Pokémon's **speed** signals the model's **cost** (fast = cheap/shallow, slow = expensive/deep). The other models (ABRA, DITTO, KADABRA, CHOMP) do different jobs and don't sit on that speed scale.

What each one does

Model	Pokémon	What it does	Cost tier	Status
ABRA	Abra	collects, stores, and models the live ladder	— (the platform)	Built
JOLTEON	Jolteon	instant team-vs-team win probability	↯ fastest / cheapest	Built
MEDICHAM	Medicham	short exact-damage rollouts; vets teams	mid	Built
SLOWKING	Slowking	deep belief-search (equilibrium play)	slowest / deepest	Research
DITTO	Ditto	builds teams vs the live meta	— (uses JOLTEON)	Built
KADABRA	Kadabra	turn-by-turn coach on your replays	— (uses CHOMP+ABRA)	Built

Model	Pokémon	What it does	Cost tier	Status
-------	---------	--------------	-----------	--------

CHOMP	Garchomp	the bring-4 / lead-2 engine you play with	— (exact damage)	Built
--------------	----------	---	------------------	-------

Only JOLTEON → MEDICHAM → SLOWKING form the fast-to-slow "cost tier" scale; the rest do their own jobs.

The acronyms. ABRA — Automated Battle Replay Analyzer. JOLTEON — Joint Odds, Ladder-Trained Expected-Outcome Network. MEDICHAM — Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves. SLOWKING — Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver. DITTO — Double-oracle Iterative Team-Tuning Optimiser. KADABRA — Key Analysis of Decisions, Advice & Better Replay Annotation.

The cost tiers. JOLTEON: microseconds (one forward pass). MEDICHAM: milliseconds (a few rollouts). SLOWKING: seconds (belief search). DITTO uses JOLTEON as its evaluator; KADABRA uses CHOMP + ABRA.

The supporting data models

Piece	What it produces
durable-ingest	every game's teams, brings, leads, revealed sets, result, ratings, bot flag, plus a per-turn stream (move order → speed, damage % per move)
dynamics	observed speed (who-moves-first, Scarf hints) and observed damage distributions per move
analyze (meta-usage)	team% / bring% / lead% / win% per species at any rating cutoff — what CHOMP reads

The one honest lesson

DITTO can **Goodhart** JOLTEON — build a team JOLTEON scores at 90% that is really ~12%. MEDICHAM's grounded rollouts catch it. That's why there's a fast guesser *and* a slower checker: **Tier-1 proposes, Tier-2 vets**. Their disagreement flags both hype (JOLTEON-high, MEDICHAM-low) and hidden gems (JOLTEON-low, MEDICHAM-high).

Who feeds whom (quick read)

- **ABRA → everything**. The store + dynamics + usage feed all models.
- **JOLTEON → DITTO**. Fast evaluator for the team search.
- **MEDICHAM → DITTO, KADABRA**. Grounded rollouts vet teams and answer "what if".
- **SLOWKING → KADABRA**. Deep lines (roadmap).
- **ABRA → CHOMP**. Usage as advisory intel; CHOMP auto-updates, so a smarter ABRA reaches your plugin with no reinstall.
- **KADABRA → ABRA**. Every reviewed game is appended back to the store (the flywheel from the coaching seat).

See the interaction chart: [architecture-diagram.svg](#).

PART 2

How the models interact



ABRA — Executive Summary

Version 1.1 · Last updated 2026-07-22 · Will Hooper

In one line

ABRA is the live-data platform for competitive Pokémon Champions: it continuously collects real ladder games and turns them into a durable dataset that feeds models of games and teams. The CHOMP bring-4 engine is one small, early consumer — proof the pipe delivers, not the purpose.

The problem

Choosing four of your six at team preview depends on what the opponent will bring. Public usage sites are stale and averaged; damage calculators see one matchup at a time. Nothing turns the *live* ladder into a model a decision engine can read and that improves as people play.

What ABRA does

- **Ingests the ladder automatically.** Public replays, pulled in parallel at ~200 games/second; the ~5,000 recent public games ingest in under a minute. A scheduled cloud job keeps it current hourly, with no operator.
- **Stores raw, analyses on top.** Every game is saved with every fact — both teams, brings, leads, observed sets, result, both ratings, and a bot flag. Any question (high-ladder only, humans only, just my games) is a filter on that store, **never** a re-download. This is the property that means the model can change forever without re-pulling.
- **Produces a usage model** — team%, bring%, lead%, win% per species, at any rating cutoff — and writes it where CHOMP reads it.

How it makes CHOMP smarter

CHOMP and ABRA are separate but connected. ABRA feeds CHOMP the ladder usage model as **advisory intel** — what the opponent will probably bring and lead. The pick itself covers the opponent's *whole six*: you cannot know which four they will bring, so CHOMP chooses the four that answer all six, and ABRA's prediction informs the read rather than gambling the bet. Because CHOMP auto-updates from its repository, a better ABRA model reaches the live plugin with no plugin change — the "gets smarter over time" loop, closed and tested.

Evidence

- Validated on **1,501 real ladder games** (2,292 human teams). The high-ladder cut (humans, 1300+) reveals distinct signal — top threats near 50% usage, win rates that separate (Kingambit 62%, Incineroar 65%).
- The top of the model independently reproduces a hand-curated threat list built from a **separate ~141,000-battle dataset** — external corroboration that the pipeline is correct.
- The ABRA→CHOMP flow-back is pinned by an automated test on a real battle: CHOMP loads the model and focuses the pick on the opponent's likely four against the user's six.

Honest limits

Revealed sets are partial (a Pokémon that never attacks reveals no moves). The model is descriptive, not a win-probability oracle. Bot and low-ladder games differ from strong play — which is exactly why they are tagged, so the consumer chooses the population.

The model family (the named cast)

The simulator is decomposed into tiers, each a Pokémon whose *speed* signals

the model's cost:

- **JOLTEON** — *Joint Odds, Ladder-Trained Expected-Outcome Network* (Tier 1, fastest): instant team-vs-team win probability. ~55% held-out vs ~49% coin flip; rarity-aware so rare picks aren't overrated. *Built*.
- **MEDICHAM** — *Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves* (Tier 2): short rollouts over CHOMP's exact damage. Rain beats sun 0.60, mirror 0.51. *Built*.
- **SLOWKING** — *Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver* (Tier 3, slowest/wisest): deep belief-search. *Scaffolded; honest research roadmap*.
- **DITTO** — *Double-oracle Iterative Team-Tuning Optimiser*: builds teams vs the live meta, usage-weighted so you get an answer to Basculegion, not Camerupt. *Built*.
- **KADABRA** — *Key Analysis of Decisions, Advice & Better Replay Annotation*: turn-by-turn coach that reconstructs each scene and flags high/low rolls. *v1 built*.
- **CHOMP** — the bring-4 / lead-2 engine you play with. *Built, separate-but-connected*.

Data now captured per game

Beyond teams/brings/leads, the extractor now records a **per-turn stream** — move order (→ real speed, Choice-Scarf detection) and exact damage % per move. 30,611 turns, 55,336 damaging hits. Raw logs are archived, so any new field is a re-parse, never a re-pull (proven: format tags backfilled onto 4,999 games with zero downloads).

The flywheel (the reason ABRA exists)

1. **Collect** live ladder games — *built and running*.
2. **Simulate** — game models from the data (JOLTEON, MEDICHAM) — *v1 built*.

3. **Optimise teams** against real meta threats (DITTO), iterating on itself — *v1 built*.
4. **Play** the best teams; CHOMP handles bring-4 / lead-2 — *built*.
5. **Feed back** — every enemy team you face (incl. every game KADABRA reviews) is auto-added.

The loop closes: more games → better simulator → better teams → more wins → more games. The most important early result is an *honest* one: DITTO can Goodhart JOLTEON (a "90%" team is really ~12%), which MEDICHAM catches — the tiered design earning its keep.

Status

v1.1 — the data platform is built and validated; Tiers 1-2 and the DITTO optimiser are built and measured; Tier 3 (SLOWKING) is scaffolded research. An interactive site (ABRA WORLD) runs JOLTEON live in the browser. CHOMP proves the pipe delivers.

Read next: [deck](#) · [white paper](#) · [simulator/ML white paper](#) · [KADABRA spec](#)

ABRA — the deck

The Pokémon that reads the whole ladder

Version 1.1 · Last updated 2026-07-22 Will Hooper

Plain words only. The math and sources live in the [white paper](#).

Slide 1 — What ABRA is

A program that watches thousands of real Pokémon games and learns what everyone is playing.

It reads public battle replays from Pokémon Showdown — the Champions Reg M-B ladder — and turns them into a live picture of the metagame. Then it hands that picture to CHOMP.

Slide 2 — CHOMP and ABRA are a team

- **CHOMP** picks your four and your lead in the 30 seconds before a battle.
- **ABRA** tells CHOMP what the ladder actually brings, so those picks are based on reality.

Two separate programs, one loop. CHOMP is the hands; ABRA is the eyes.

Slide 3 — Why this matters

To choose your four well, you need to know what your opponent will bring. A

static usage list from a website is weeks old and averaged over everyone. ABRA reads the *current* ladder and updates itself.

Slide 4 — It reads thousands of games, fast

The replay site lets anyone download public games. ABRA grabs them in parallel — about **200 games a second** — so the last few thousand games ingest in under a minute.

There are roughly **5,000** recent public games available at any moment, and new ones upload constantly.

Slide 5 — The one rule that saves us from ever redoing it

Store everything, decide later.

For every game, ABRA saves *all* the facts — both teams, what each side actually brought, what they led, the moves and items we saw, who won, both players' ratings, and whether a player was a bot.

Because everything is stored, any question — "show me only strong players," "hide the bots," "just my games" — is answered instantly from the saved data.

We never have to download the games again.

Slide 6 — Bots and rating

The ladder has automated **bot** accounts mixed in with real people. ABRA flags them. It also saves each player's rating. So you can look at the *whole* ladder, or *only humans above 1300*, or anything in between — your choice, no re-downloading.

That filter matters: at high ladder the picture sharpens — the top threats show up on half of all teams, and some win far more than others.

Slide 7 — Keeping your games separate

Your own games are just a filter: any game with your Showdown name. So ABRA can show you the whole ladder **and** your personal record from the same pile of data.

Changed your Showdown name? Add the new name to a short list once. Nothing is lost, nothing re-pulled. No accounts, ever.

Slide 8 — It works for anyone

The ladder picture belongs to everyone. Your personal view is just your name. So a friend points ABRA at their own name and gets their own analysis from the same shared data. One tool, many users, no sign-ups.

Slide 9 — It runs itself

A scheduled job in the cloud wakes up every hour, grabs the newest games, updates the model, and saves it. You don't run anything. The picture stays current on its own — and because CHOMP updates itself too, your live plugin quietly gets smarter while you play.

Slide 10 — Now it reads *inside* the games too

ABRA no longer just notes who was on each team. For every game it now records, turn by turn, **who moved first** (that reveals real speed — even a hidden Choice Scarf) and **exactly how much damage each move did**.

Thousands of games of this add up to a real picture of the physics: we know Basculegion's Wave Crash takes about 62% on average, and we can tell when someone got a lucky roll.

Slide 11 — The cast of models

Each part of the plan is a Pokémon, and the Pokémon's **speed tells you how expensive the model is**:

- ⚡ **JOLTEON** (fastest) — instant "who's favoured?" between two teams. Built.
 - 🏥 **MEDICHAM** (medium) — actually plays the matchup out a few turns with real damage math. Built.
 - 🧠 **SLOWKING** (slow, wise) — the deep thinker that plans around what the opponent is hiding. Still being built.
 - 🔄 **DITTO** — builds teams that beat the live ladder, making sure you answer the *common* threats. Built.
 - 🎙️ **KADABRA** — the coach that replays your game with you and points out the better move. Built.
 - 🎮 **CHOMP** — the engine you actually play with, picking your 4 and your lead.
-

Slide 12 — The most honest result we have

When DITTO built a team to beat JOLTEON's scoring, JOLTEON said it would win **90%** of the time. Then MEDICHAM played it out for real and said **12%**. JOLTEON had been fooled — it over-trusted a few rare Pokémon. That's not a bug we hid; it's *why* we have a fast guesser and a slower checker. The disagreement between them is a feature: it flags both hype and hidden gems.

Slide 13 — ABRA WORLD (the fun part)

There's a little snowy town website where every model is its own building. You can pick two teams from sprites and watch JOLTEON — the *real* model — call the odds right in your browser, with confetti when the battle resolves. It's `web/index.html`.

Slide 14 — Honest limits

- A Pokémon that never attacked in a replay tells us nothing about its moves — so what we know about each set is partial.
 - These are **descriptions** of what people play, not a prediction of who wins.
 - Low-ladder and bot games look different from strong play — which is exactly why we tag them and let you choose.
-

Slide 15 — Read more

The white paper — the data model, the ingest, the math, the sources:

[ABRA-whitepaper.md](#)

The simulator / ML white paper — the machine learning behind the model family: [ABRA-simulator-whitepaper.md](#)

The coach spec — how KADABRA teaches: [KADABRA-coach-spec.md](#)

The changelog — what changed and why: [CHANGELOG.md](#)

How Predictable Is Pokémon Champions? A Ratings-and-Models Study

Measuring the irreducible variance of a competitive VGC metagame

Version 1.0 · 2026-07-23 · Will Hooper · ABRA

*An empirical study, run entirely over ABRA's durable store of real Champions (Reg M-B) ladder games. It asks a question every competitor feels but rarely measures: **when the better team or the better player shows up, how often do they actually win?** The answer bounds what any pre-game model — JOLTEON, a calibrated CHOMP, anything — can achieve, and it reframes the models' "only ~56%" not as weakness but as brushing against a hard ceiling the game itself imposes. Reproduce with `engine/predictability.py`.*

0. The CHOMP-JOLTEON-win% dynamic, in one place

Three things people conflate, separated:

- **JOLTEON predicts the winner.** Given two team sheets, it outputs a calibrated win *probability*. It is a forecaster. Its accuracy is ~55%.
- **CHOMP decides what to do.** Given your six and theirs, it picks the four to bring and two to lead by exact damage. It is a decision engine. It does not forecast a winner at all.

- **Winning percentage is the ceiling both live under.** This study shows the game only lets the better side win ~55%, rising to ~58% with a big edge. That number is set by the game's variance, not by any model.

How they relate:

1. **As predictors, CHOMP and JOLTEON would be near-identical in accuracy (~55-56%)** — because both read only the team sheet, and the sheet's predictive power is capped at the ceiling. They differ in *how* they reach it (JOLTEON = learned species strengths; CHOMP = exact damage), so they make **different mistakes**, which is why you combine them (CHOMP's damage score as a feature in JOLTEON) rather than pick one.
2. **CHOMP raises your win% a different way than JOLTEON measures it.** JOLTEON tells you the pre-game odds; it cannot change them. CHOMP acts *inside* the game — bringing the right four — which moves the part of the outcome the forecaster can't see. So CHOMP improves winning *by deciding well*, not by predicting well.
3. **The ceiling is the whole point.** Because pre-game prediction caps at ~55-58%, the leverage is downstream of the sheet: in-game decisions (CHOMP now; MEDICHAM/SLOWKING deeper) and team building against the meta (DITTO). Prediction is nearly maxed out; *decisions and construction are not*.

The rest of this study is the evidence for that ceiling.

1. Why this matters

Every model in ABRA that predicts a *winner before the game* — JOLTEON today, a calibrated CHOMP tomorrow — lands around 55-57% accuracy, and we have reported that honestly throughout. The natural worry is that the models are simply weak. This study tests the alternative hypothesis: that **55-57% is close to the maximum any pre-game predictor can reach**, because Champions is a high-variance game in which the better side wins

only slightly more than half the time. If true, chasing a team-sheet model to 70% is chasing a number the game does not contain, and the real leverage is elsewhere (in-game decisions, team building against the meta) — exactly where ABRA's other models aim.

We attack it two ways that need no model at all, then check the model against them.

2. Data

All finished games in the durable store (4,998 unique replays). For the ratings analysis we keep the **2,337 human-vs-human games where both players have a Showdown rating**. Ratings are read directly from the replay `|player|` lines; no modelling is involved. For the JOLTEON analysis we use the same humans-only temporal split as the model's own evaluation.

3. Result 1 — how often does the better *player* win?

Across 2,318 games with unequal ratings, **the higher-rated player won just 55.0% of the time**. Broken down by the size of the rating gap, and compared to what the Elo formula $p = 1/(1+10^{(-\Delta/400)})$ predicts:

Rating gap (Δ)	Games	Higher-rated actually wins	Elo predicts
0-25	592	49.3%	51.8%
25-50	547	54.8%	55.3%
50-100	1034	57.9%	60.5%
100-200	164	57.9%	65.1%

Two things stand out. First, **at small rating gaps the game is a coin flip** — a 0-25 point edge wins 49.3%, statistically indistinguishable from chance. Second, **even a large edge tops out around 58%**, and it does so *below* the

Elo prediction: a 100–200 point gap "should" win ~65% but wins only ~58%. Champions compresses skill. The better player is favoured, but the variance — damage rolls, 50/50 protects, speed ties, team-matchup luck — repeatedly overrules them.

4. Result 2 — how well-calibrated is JOLTEON, and where is the ceiling?

On the held-out temporal split (513 games, humans only), JOLTEON scores **54.6% accuracy, Brier 0.252, log-loss 0.699** — right where the ratings result says a pre-game predictor should live. Its probabilities are reasonably calibrated: when it says 55–65%, the favoured team wins ~60%; when it says under 35%, they win ~32%. It is honest about its own uncertainty rather than overconfident.

The telling number is the **ceiling**. JOLTEON's most extreme honest probability on this data is about **81%**, and it reaches that only for a handful of lopsided team-sheet mismatches. For the bulk of real games — two competent teams — it correctly declines to be confident, because the game does not permit confidence. This is the same wall the ratings analysis hit from the other side.

5. Result 3 — CHOMP as a predictor (method and expectation)

CHOMP already computes an exact-damage matchup score (`teamVs / bring4`) for the best-4 of one six against the other. Turning that into a win probability is a one-line calibration: $P(\text{win}) = \sigma(a \cdot (\text{score}_1 - \text{score}_2) + b)$, with `a`, `b` fit against the stored outcomes — a logistic on data we hold. `engine/chomp-predict.js` emits the per-game score difference for this fit.

We report the **method** rather than a full backtest number here for an honest reason: computing CHOMP's score requires the full damage engine, and over thousands of games its memory footprint exceeds this analysis sandbox's ceiling (it runs in small batches; a full run needs a roomier machine). But the

expected result is not in doubt, and Results 1-2 tell us why: a calibrated CHOMP predictor is another **pre-game team-sheet model**, so it is bounded by the same ~55-58% ceiling. Grounding the prediction in exact damage instead of learned species strengths changes *how* it reaches the ceiling, not *where* the ceiling is. The genuinely valuable use of CHOMP's score is therefore not as a rival predictor but as a **feature inside JOLTEON** (§7).

6. Synthesis — the predictability ceiling of Champions

Three independent measurements agree:

- the **better player** wins ~55% (58% with a big rating edge),
- the **better team** (JOLTEON) is called correctly ~55%, calibrated, with a practical confidence ceiling ~81% reached only on rare mismatches,
- a damage-grounded predictor (CHOMP) is bounded by the same wall by construction.

So the answer to "*how often does the better side win?*" is: **only a little more than half — around 55%, rising to ~58-60% when the edge is large.**

That is the irreducible variance of the format. It is not a flaw in the models; it is a property of the game, and the models are already near it.

What about the better *play* (per turn)?

This study measures the better *team* and the better *player*. Measuring how often the better *in-game move* wins is harder: it needs a solver that can say which move was better, i.e. SLOWKING's belief search. We cannot yet quantify it from data alone. But the team/player ceiling **bounds** it — in a game where the better player wins 55%, individual correct plays swing games by less, which is exactly why a coach (KADABRA/ALAKAZAM) should talk in terms of *shifting win probability at the margin*, not guaranteeing outcomes.

7. Implications for ABRA

- **Stop chasing pre-game accuracy past ~58%.** It is not there. JOLTEON's rarity-aware ~56% is near-optimal for a team-sheet model; the honest move is calibration, not more accuracy.
- **The leverage is downstream of the sheet.** In-game decisions (MEDICHAM, SLOWKING) and team-building against the meta (DITTO) act on the part of the outcome the sheet cannot see — which is most of it.
- **Feed CHOMP's damage score into JOLTEON as a feature,** not as a competitor. It adds information a learned-identity model lacks, and it is the most principled way to squeeze the last point or two out of the pre-game predictor before the ceiling stops everyone.
- **Coaching should be probabilistic.** Because even large edges only win ~58%, KADABRA's advice is correctly framed as "this play was +X% win probability," not "this play wins."

8. Honest limits

The ratings are Showdown ladder Elo, which is noisy and non-stationary; the humans-only rated sample is 2,337 games from a ~2-day window, so the rating-gap bins (especially 100-200) are small and their confidence intervals are wide. The direction — a low, slowly-rising better-side win rate well under Elo's prediction — is robust across bins and matches the independent JOLTEON result, but exact percentages will move as the store grows (and the recency-weighted models will track that drift). The CHOMP backtest number is deferred, not claimed.

9. References

1. Elo, A. (1978). *The Rating of Chessplayers, Past and Present*.
2. ABRA — [simulator white paper](#) (the ~55-57% ceiling, derived); [SLOWKING white paper](#) (why per-move "better play" needs a solver).

3. Reproduce: `engine/predictability.py`, `engine/chomp-predict.js`.

Companion. [Cheat sheet](#) · [Architecture notes](#) · [Changelog](#)

Modelling a Live Metagame from Public Replays

A technical description of ABRA, the Automated Battle Replay Analyzer

Version 1.0 · Last updated 2026-07-22 Will Hooper · ABRA

This is a living document, updated in the same pass as any change to the code. New information is appended; a prior conclusion is not silently rewritten. See `CHANGELOG.md`.

Abstract

ABRA is a live-data platform for competitive Pokémon Champions (Reg M-B). It continuously ingests public battle replays from Pokémon Showdown, extracts the durable facts of every game, and stores them as a growing dataset — the foundation for **modelling games and teams**. The near-term model is a metagame usage model; the larger goal is a simulator that learns which teams and which in-game choices win. This paper states the data model, the ingest architecture, the first analysis, and the limits. The governing design rule — *store raw, analyse on top* — is defended as the property that lets every model built on the data change without ever re-fetching. A team-preview engine (CHOMP) is one small, early consumer of the output; it is a proof of delivery, not the purpose.

1. The problem

Competitive VGC has many hard questions — which six to build against the meta, which four to bring, how a matchup tends to play out. Answering any

of them well needs data on how the ladder *actually* plays, kept current as it changes. Existing tools give static usage snapshots (Pikalytics) or per-game damage math (calculators); none continuously collects live games into a durable dataset that downstream models — team optimisers, game simulators, decision engines — can all build on. ABRA is that collection-and-storage layer. The bring-4 engine (CHOMP) is the first small model on top.

2. The data source

Pokémon Showdown exposes public replays through a stable API:

- `search.json?format=<id>` — recent public replays for a format, paginated (~51/page, ~100 pages).
- `search.json?user=<name>` — a specific player's public replays.
- `<id>.log` — the full battle protocol for one replay.

Every replay is a public artifact already attached to public usernames; ABRA reads nothing private and creates no accounts (see `SECURITY.md`).

3. The record — what we store per game

The extractor (`engine/durable-ingest.js`, `extract()`) turns one `.log` into one durable record:

Field	Meaning
<code>id, date</code>	replay id and upload time
<code>p1, p2</code>	<code>{name, rating, bot}</code> per player — rating from the <code>\ player\ </code> line, <code>bot</code> by name pattern
<code>six.p1/p2</code>	the revealed team of six (<code>\ poke\ </code> lines at preview)
<code>brought.p1/p2</code>	the Pokémon actually sent out (<code>\ switch\ </code> lines)
<code>lead.p1/p2</code>	the first two sent out

Field	Meaning
sets	per species, the moves / item / ability the replay <i>revealed</i>
winner	the winning name (<code>\ win\ </code>)

This is deliberately richer than any single analysis needs. That richness is the insurance behind the design rule in §5.

4. Ingest architecture

- **Concurrent fetch.** A fixed-size promise pool fetches logs in parallel. Measured throughput is ~200 logs/sec, so the full ~5,000 recent public replays ingest in about 25–30 seconds.
- **Incremental and idempotent.** The store is append-only JSON Lines keyed by replay id. Each run reads existing ids, requests recent pages, and appends only games not already present. Re-running never duplicates and never re-fetches stored games.
- **Never double-counted.** A battle has one fixed Showdown replay id, so a game uploaded by *both* players — or one you later review in KADABRA, or pull via your own username — is the same id and is ignored if already stored. Every ingest path (ladder scrape, personal pull, coaching review) dedups against the full store, and every analysis loader (`jolteon`, `dynamics`, `analyze`) also dedups by id at read time, so a stray duplicate line can never inflate a count.
- **Continuous collection.** A scheduled GitHub Action runs the ingest hourly and commits the refreshed store and model, so the metagame model tracks the live ladder with no operator.

5. The governing rule — store raw, analyse on top

Every analysis and every filter runs over the stored records, not over the network. Concretely:

- The rating cutoff, the humans-only filter, and the "just my games" filter

are all computed from the same store at read time.

- Changing how games are *placed* (a different rating tier, excluding bots, a new archetype tag) is a re-computation, **never** a re-pull.

The alternative — deciding the population at fetch time — would force a re-fetch every time the question changed. Storing raw makes the fetch a one-time cost and the analysis free to evolve. This is the single most important property of the system.

6. The usage model

For a chosen population, `engine/analyze.js` computes per species:

```
teamRate  = teams containing the species      / teams
bringRate = games the species was brought     / games it was on the team
leadRate  = games the species was led         / games it was on the team
winRate   = games won when the species was brought / games it was brought
```

Species below a small sample floor ($n \geq 8$) are dropped. The result is written to `data/meta-usage.json`, the model CHOMP reads.

6.1 A validated result

On 1,501 stored games (2,292 human teams), the raw ladder and the high-ladder cut (humans, rating ≥ 1300 , 530 teams) differ meaningfully — the high-ladder cut lifts top-threat usage toward 50% and separates win rates (e.g. Kingambit 62%, Incineroar 65%). The top of the raw table independently reproduces a hand-curated threat list built from a separate ~141k-battle dataset (Basculegion, Whimsicott, Garchomp, Kingambit), which is external corroboration that the pipeline is correct.

7. Feeding CHOMP

CHOMP is a separate project — only the bring-4 / lead-2 engine. ABRA feeds it the ladder usage model (`meta-usage.json`) as **advisory intel**: what the opponent will *probably* bring and lead. CHOMP can sharpen that into a matchup-aware prediction — their best four against your six — but the **pick**

itself covers the opponent's whole six. A player cannot know which four the opponent will bring, so CHOMP optimises for coverage over all six rather than gambling on a predicted four; ABRA's prediction informs the read, not the bet. Because CHOMP auto-updates from its repository, an improved ABRA model reaches the live plugin with no plugin change. This is the "gets smarter over time" loop, made concrete, and it is pinned by `CHOMP/tests/test-meta-flow.js`.

8. The flywheel — where this is going

ABRA's data platform is stage one of a self-improving loop. Each stage feeds the next, and the last stage feeds the first:

1. **Collect** — continuously ingest live ladder games into the durable store. *(built and running)*
2. **Simulate** — build a near-perfect game simulator from that data: given two teams, model how the matchup tends to play out. *(roadmap)*
3. **Optimise teams** — run hypothetical teams against the real metagame threats in the simulator and let the search iterate on itself, converging on sixes that beat what the ladder actually brings. *(roadmap; the meta threat model that feeds it is built)*
4. **Play** — take an optimised team onto the ladder; CHOMP handles the bring-4 / lead-2 in each game. *(CHOMP built)*
5. **Feed back** — every opponent team you face is auto-added to the store, so playing the game grows the dataset. *(the ingest + store support this directly; the auto-add-on-play hook is roadmap)*

The loop closes: more games → a better simulator → better teams → more wins → more games. The value is not any single model but the flywheel — the system gets stronger the more it is used. This is why the design rule (*store raw, analyse on top*, §5) matters so much: every model in the loop is built on the same growing dataset, and none of them ever forces a re-pull.

Honest status. Stage 1 is built and validated (§4, §6.1). Stages 2 and 3 — the simulator and the self-iterating team optimiser — are not yet built; they

are the reason ABRA exists and the next work. CHOMP (stage 4) is live. The paper does not claim the flywheel is spinning yet; it claims the foundation that makes it possible is in place.

9. Limitations

1. **Revealed sets are partial.** A Pokémon that never attacked reveals no moves; items/abilities appear only when they trigger. Set inference is a lower bound, not the full set.
2. **Descriptive, not predictive.** Usage and win rates describe the population; they are not a win-probability model.
3. **Population choice matters.** Bot and low-ladder games differ from high-ladder play. ABRA tags rating and bots so the consumer chooses the population; it does not decide for them.
4. **Store growth.** The append-only store grows without bound; pruning by date is future work.

10. References

1. Pokémon Showdown replay API — replay.pokemonshowdown.com.
2. [docs/reg-mb-threat-list.md](#) (CHOMP) — the independently curated threat list used for validation.
3. Keep a Changelog; Semantic Versioning; ASD-STE100; Diátaxis.

Companion documents. [Slide deck](#) · [Technical documentation](#) · [Changelog](#)

Learning a Battle Simulator from Logged Replays

Machine-learning foundations for a tiered VGC game model

Version 1.1 · Last updated 2026-07-22 Will Hooper · ABRA

Build status (v1.1): Tiers 1 (JOLTEON) and 2 (MEDICHAM) and the DITTO outer loop are built, tested, and measured (see §12). The per-turn dynamics model is built. Tier 3 (SLOWKING) is scaffolded and honestly roadmap. Numbers below are from real runs.

A research white paper on the modelling problem behind ABRA's stages 2–3 (the simulator and the team optimiser). It states the problem formally, surveys the relevant machine-learning literature, derives the math for each modelling tier, and gives an honest feasibility assessment. Living document; updated in the same pass as the code it describes.

Abstract

ABRA collects a growing corpus of Pokémon Champions (Reg M-B) battle replays. We want to turn that corpus into a *simulator*: a model that, given two teams, predicts how their game tends to resolve, so that hypothetical teams can be optimised against the live metagame without playing thousands of real games. The underlying game — a two-player, zero-sum, simultaneous-move, stochastic game of imperfect information — is implemented by a closed engine we cannot query. We therefore treat the problem as **learning a model from logged data**. This paper formalises the game, decomposes the simulator into three modelling tiers of increasing fidelity and cost (an outcome model, a hybrid rollout model, and a learned dynamics model),

derives the estimator and its failure modes for each, frames team optimisation as the outer loop of a self-improving flywheel, and specifies an evaluation protocol that is honest about the irreducible variance of the domain. We ground each tier in the 2025 Pokémon-AI literature — PokéChamp, Metamon, and VGC-Bench — and conclude with a feasibility assessment: tier 1 is immediately achievable on ABRA's data, tier 2 reuses the existing CHOMP damage engine, and tier 3 is a genuine research effort that recent work shows to be possible but expensive.

1. Introduction

A competitive VGC player faces three nested questions: which six Pokémon to build, which four to bring, and how to play the resulting game. CHOMP answers the middle question with exact damage math. The hardest lever, and the one with the most upside, is the first: *which six beats the current metagame*. Answering it by trial on the ladder is slow and high-variance. The alternative is to **simulate** — to evaluate a candidate team against the meta in software, iterate, and only then play. This paper is about building that simulator from the data ABRA already collects.

The central obstacle is that the Champions battle engine is **closed**. Unlike the standard Smogon formats served by `pokemon-showdown`'s open-source simulator — which the `poke-env` library exposes to RL agents — the Champions ruleset and its stat system are not available as a queryable environment. We cannot roll the true dynamics forward for a hypothetical state. What we *do* have is a large and growing set of **observed trajectories**: real games, each a sequence of states and joint actions ending in a win or loss. Learning a usable model from such logs, with no ability to interact with the true environment, is exactly the setting of **offline (batch) reinforcement learning** and **model learning / system identification**. This is not a toy analogy: Metamon (§9) reconstructs 5M+ trajectories from human Showdown replays and trains competitive agents on them offline.

2. The game, formally

A single VGC battle is a **partially observable stochastic game (POSG)**, and specifically a two-player, zero-sum, *simultaneous-move* Markov game with imperfect information. We fix notation.

- **State** $s \in S$. The full state comprises both teams' species, sets (moves, item, ability, the Champions SP spread), current HP, status, stat stages, field conditions (weather, terrain, screens, Trick Room), and which Pokémon are active. Level 50; no Tera in Champions.
- **Players** $i \in \{1, 2\}$.
- **Actions**. At each turn both players choose *simultaneously* from a legal set $A_i(s)$ — for each of two active Pokémon, a move (with a target) or a switch. The **joint action** is $a = (a_1, a_2)$. Team preview is a special first move: each player chooses an ordered bring of four from six, $A_i = \{\text{ordered 4-subsets of the six}\}$, $|A_i| = 6 \cdot 5 \cdot 4 \cdot 3 = 360$.
- **Transition kernel** $T(s' \mid s, a)$. Stochastic: damage rolls (16 discrete outcomes, 85–100%), accuracy, secondary-effect procs, critical hits, speed ties. This kernel is the *engine*, and it is what a full simulator must reproduce.
- **Reward**. Zero-sum and terminal: $r = +1$ for the winner, -1 for the loser; 0 elsewhere. Define the **value** of a state to player 1 as $V(s) = E[r \mid s, \text{play}]$ under a solution concept.
- **Observation**. Player i sees $o_i = o_i(s)$: their own team fully, and of the opponent only what has been revealed (the six species at preview; each move/item/ability only once used or triggered). This partial observability is first-class — it is why opponent modelling is a distinct problem, and why belief states matter.

Two structural facts shape everything downstream:

1. **Simultaneous moves $\Rightarrow$ mixed strategies**. Because both players commit without seeing the other's choice, the per-turn subgame is a matrix game whose optimal solution is generally a *mixed* (randomised) Nash equilibrium, not a single best move. A simulator that assumes

deterministic opponent play is systematically wrong at exactly the decision points that matter.

2. **Imperfect information $\Rightarrow$ belief states.** The relevant object is not s but a distribution over states consistent with the observation history — a belief $b(s)$. Solving the game exactly is the province of **counterfactual regret minimisation (CFR)** and its deep variants; approximations are unavoidable at VGC's scale.

The team-building question sits one level above the battle: it is a **Bayesian game** in which a player commits to a team before knowing the opponent's, and payoffs are the expected battle values integrated over the opponent distribution the metagame induces.

3. Three modelling tiers

A single "simulator" of full fidelity is neither necessary nor tractable for every use. We decompose it into three tiers, coarse-to-fine, and choose per use (broad search vs. finalist vetting). This is the standard model-based pattern: cheap approximate models for planning breadth, expensive accurate models for depth.

Tier	Object learned	Query	Cost	Primary use
1. JOLTEON (outcome)	$P(\psi(\text{win} \mid \text{team_A}, \text{team_B}))$	one forward pass	$\sim \mu\text{s}$ (fastest)	search over thousands of teams
2. MEDICHAM (hybrid rollout)	CHOMP damage + learned policies	a few short rollouts	$\sim \text{ms}$	ranking finalist teams

Tier	Object learned	Query	Cost	Primary use
3. SLOWKING (learned dynamics)	$T\theta(s' s, a)$ + value/policy	belief search / payout	$\sim s$ (slowest)	deep vetting, a matchup

The rest of the paper derives each.

3.1 The model family, named

Each model gets its own name, in the CHOMP / ABRA tradition — and the Pokémon's *speed* signals the model's cost, fast to slow:

- **JOLTEON** — *Joint Odds, Ladder-Trained Expected-Outcome Network* (Tier 1). The **fastest** model (Jolteon is the quickest of the cast): an instant win-probability call between two teams, from Bradley-Terry strengths plus CHOMP-derived damage/coverage features (§4.3.1). One forward pass, microseconds — fast enough to score a whole team search.
- **MEDICHAM** — *Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves* (Tier 2). Middle of the pack, in name and in speed: it plays a matchup out a few turns using **CHOMP's exact damage engine** and behaviour-cloned move priors. Grounded, approximate, mid-cost.
- **SLOWKING** — *Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver* (Tier 3). The **slowest** and deepest: a slow but wise brain that searches over belief states on a learned dynamics model toward equilibrium play. Slow on purpose — you run it only on finalists.
- **DITTO** — *Double-oracle Iterative Team-Tuning Optimiser* (§8, the outer loop). It tries team after team against the meta and transforms toward the strongest six, the way Ditto tries every form.

CHOMP (the bring-4 engine) and the coach are *consumers* of these models, not models themselves. The naming makes the pipeline legible and the

speeds honest: ABRA feeds MEDICHAM and JOLTEON; those and SLOWKING feed DITTO; DITTO's teams and SLOWKING's lines reach the player through CHOMP and the coach; the games played feed ABRA. Fast Pokémon do the cheap, broad work; the slow one does the deep work.

4. Tier 1 — JOLTEON, the outcome model

4.1 Objective

Learn a function that maps an ordered pair of teams to a win probability: $P_\psi(y = 1 \mid x_A, x_B)$, where $y = 1$ iff team A wins and x is a feature encoding of a team. This is supervised binary classification on the log: each stored game contributes one labelled example (and its mirror, with the label flipped, to enforce the antisymmetry $P(A \text{ beats } B) = 1 - P(B \text{ beats } A)$).

4.2 The Bradley-Terry / Elo backbone

The natural, well-founded baseline treats each team (or, more usefully, each *component*) as having a latent strength and models paired-comparison outcomes with the **Bradley-Terry** model:

$$P(A \text{ beats } B) = \sigma(\beta_A - \beta_B), \quad \sigma(z) = 1 / (1 + e^{-z}).$$

Fitting β by maximum likelihood over the observed win/loss pairs is exactly logistic regression; online, the same model with a specific update rule is **Elo**. Two refinements matter for VGC:

- **Teams are compositional.** Rather than a strength per *team* (there are astronomically many), parameterise strength as a sum over the team's members and interactions: $\beta_A = \sum_{p \in A} w_p + \sum_{\{p, q \in A\}} u_{\{pq\}}$, learning a per-species main effect w_p and pairwise synergy $u_{\{pq\}}$. This is a factorisation-machine view and it generalises to unseen teams built from seen parts — the property we need, since we will score *hypothetical* teams.
- **Matchup structure (advantage cycles).** Pokémon has rock-paper-scissors matchups: strength is not fully captured by a single scalar. The

Blade-Chest and low-rank bilinear extensions add a vector per team and score $\beta_A - \beta_B + \langle c_A, m_B \rangle - \langle c_B, m_A \rangle$, which can represent non-transitive advantage (A beats B beats C beats A). Including this term is what lets the model express "this team preys on the current meta" rather than "this team is generically strong."

4.3 Features

x should be a fixed-length encoding of a team invariant to Pokémon order: a multi-hot species vector (dimension = legal roster), augmented with observed-set summaries (lead rate, common item, revealed moves aggregated from ABRA's `sets` field), archetype tags (weather, Trick Room, Tailwind), and speed-tier and type-coverage summary statistics. A neural encoder (a small permutation-invariant DeepSets/Transformer over the six member embeddings) is the expressive upgrade once the linear model saturates.

4.3.1 CHOMP's threat scoring as damage-grounded features

Species identity alone is a weak feature — two teams with the same six can play very differently, and what actually decides a matchup is *who KOs whom*. CHOMP already computes exactly this, and its output is the strongest feature JOLTEON can eat. CHOMP scores threats with the real Gen-9 damage pipeline on the Champions SP stat system and reports, for an attacker-defender pair, **pKO** — the fraction of the 16 damage rolls (85–100%) that knock out — combined with a speed check (does the attacker move first). Its bring-4 chooses the four that maximise KO-pressure/coverage across the opponent's six.

For JOLTEON, build the **coverage matrix** K where $K_{\{pq\}} = pKO(p \rightarrow q)$ over every attacker p in team A and defender q in team B (and its transpose for $B \rightarrow A$), plus the speed-order mask. Summaries of K — how many of B's six A can OHKO, the best-case and worst-case coverage, symmetric speed control — become the features that carry most of JOLTEON's signal, because they are the real mechanics of the matchup rather than a usage proxy. This is grey-box modelling: CHOMP supplies the physics of a single exchange;

JOLTEON learns how those exchanges aggregate into a game outcome across thousands of real results. CHOMP's own validation (its white paper, validation report, and referee report) is therefore part of JOLTEON's foundation — the feature generator is already tested.

4.4 Estimation and calibration

Maximise the regularised conditional log-likelihood:

$$\psi^* = \operatorname{argmax}_{\psi} \sum_g [y_g \log P\psi(x_{\{A,g\}}, x_{\{B,g\}}) + (1-y_g) \log(1 - P\psi(\cdot))] - \lambda \|\psi\|^2.$$

A win **probability** is only useful if it is *calibrated* — when the model says 70%, the team should win ~70% of the time. We measure calibration with the **Brier score** $(1/N) \sum (P\psi - y)^2$ and reliability diagrams, and correct residual miscalibration with **Platt scaling** or isotonic regression on a held-out split. Calibration, not raw accuracy, is what the team optimiser consumes, because it integrates these probabilities over the meta.

4.5 What tier 1 can and cannot do

It answers "does team A tend to beat team B?" from data, generalises to unseen teams via compositional features, and runs fast enough to score a whole search. It does **not** know *why* — it has no notion of turns, and it cannot tell you the line of play. It is a value function without a model. That is exactly enough for the outer optimisation loop and nothing more.

5. Tier 3 — SLOWKING, learning the dynamics (the true "reconstruction")

Tier 3 is what "reconstruct the simulator" literally means: learn the transition kernel $T\theta(s' | s, a)$ and a policy/value on top, from logged transitions.

5.1 The dynamics model

Each replay is a sequence $(s_0, a_0, s_1, a_1, \dots, s_H, r)$. Every consecutive triple (s_t, a_t, s_{t+1}) is a supervised example for a **world model**. Fit by

maximum likelihood:

$$\theta^* = \operatorname{argmax}_{\theta} \sum_{\{t\}} \log T\theta(s_{\{t+1\}} \mid s_t, a_t).$$

Because the true kernel factorises (each active Pokémon's move resolves largely independently given speed order), a structured model predicting damage-roll distributions, status procs, and stat changes per move is far more sample-efficient than a monolithic next-state predictor. Damage is *already* known in closed form (CHOMP's engine); the learned part is the residual — accuracy, secondary effects, switching outcomes, and the opponent's hidden set. This is a **grey-box** system identification problem: known physics plus a learned residual, which needs far less data than a black-box learner.

5.2 Partial observability

We never observe s ; we observe o and must maintain a **belief** $b_t = \Pr(s_t \mid o_{\{ \leq t \}}, a_{\{ < t \}})$. The opponent's hidden set is the main latent variable. ABRA's usage model supplies the **prior** $\Pr(\text{set} \mid \text{species})$ (how the ladder builds each Pokémon), and each revealed move/item is a Bayesian update. A particle filter over sampled opponent sets, each rolled forward through $T\theta$, is the standard tractable representation. Note the clean division of labour: ABRA (data) supplies the prior; the dynamics model supplies the update.

5.3 Solving the game on the learned model — search over beliefs, not states

With $T\theta$ and beliefs, the game must be searched. Because the information is imperfect, the correct object is **not** a tree over states but a search over **public belief states (PBS)** — distributions over what each side could hold given everything commonly observed. This is the central idea of **ReBeL** (Recursive Belief-based Learning), which combines deep RL with search over PBSs and *provably converges to a Nash equilibrium* in two-player zero-sum imperfect-information games; in the perfect-information limit it reduces to AlphaZero. **Player of Games** and **Student of Games** later unified perfect- and imperfect-information search into one algorithm. This family — RL +

search over beliefs — is the principled target for a Pokémon solver, and it subsumes the two simpler options: a per-turn **matrix-game Nash** (a small linear program at each node) with **minimax/expectimax** over joint actions, which is exactly PokéChamp's architecture (LLM samples actions, models the opponent, estimates leaf values; Elo 1300–1500); and the search-free route of a **policy/value network learned offline**, as Metamon does via behavioural cloning plus offline RL and AlphaStar Unplugged does at scale.

5.4 Why offline RL is hard here (and how the literature copes)

Learning a policy or value purely from logs suffers **distributional shift**: the model is queried at state-action pairs the logging players never took, where its estimates are unconstrained and optimistic. The canonical fixes are **conservatism** — Conservative Q-Learning (CQL) penalises out-of-distribution action values; Implicit Q-Learning (IQL) avoids querying unseen actions entirely — and **return-conditioned supervised learning** — the Decision Transformer casts offline RL as sequence modelling, which is the family Metamon uses. Off-policy **evaluation** (weighted importance sampling, doubly-robust estimators) lets us score a policy from logs without deploying it, with variance that grows with policy divergence. The blunt honest point: without environment access, every tier-3 estimate is an extrapolation, and its trustworthiness must be *measured*, not assumed.

5.5 You do not simulate every path — the policy prunes the tree

The intuition that "*you would not simulate all the paths, machine learning just chooses the few viable ones*" is exactly how modern game AI is made tractable, and it has a precise form. The branching factor of a turn is large (each of two actives can move-with-target or switch), and the tree explodes geometrically with depth. AlphaZero tames this with a **policy prior** that biases the search toward promising actions; for genuinely large action spaces the fix is to **sample** rather than enumerate: **Sampled MuZero** expands only a subset of actions drawn from the prior, **AlphaStar** restricts StarCraft's enormous action space to a handful of policy-sampled actions per node, and

Gumbel MuZero makes this sampling near-optimal with few simulations. PokéChamp is the Pokémon instance: the LLM proposes only the plausible moves, so the minimax tree never touches the overwhelming majority of legal-but-pointless lines.

For ABRA this means the learned policy is not a luxury on top of the simulator — it is what makes the simulator usable. A well-trained policy assigns almost all probability to the two or three viable options in a position, so search explores a tree with an effective branching factor of a few, not a few dozen. The cost of a "near-perfect" playout is therefore governed by the *policy's* quality, not by the raw size of the game tree. This also answers a practical worry: we never need to model every interaction explicitly by hand — a policy trained on enough real games learns which interactions matter and silently prunes the rest.

6. One model, many queries — playing and testing teams from a shared core

Is the team-tester the same model as the game-simulator? They should share a core. The AlphaZero / MuZero pattern is a single network with a shared representation and two heads — a **value** head $V_\phi(\text{belief})$ and a **policy** head $\pi_\phi(\text{action} \mid \text{belief})$. That one core answers both of ABRA's questions:

- **Testing teams against the meta** is the value head, marginalised over play and over the meta distribution: $E_{\{o \sim D\}}[V_\phi(\text{team}, o)]$ (§7). No game needs to be rolled out turn-by-turn to *rank* teams if the value head is calibrated — this is precisely tier 1, and it can be **distilled** from the full model into a fast outcome head so that team search stays cheap.
- **Giving the player the line** is the policy head plus search: from the current belief, the model returns the best move(s) — the very thing the coach surfaces. Because the model has learned how the mechanics and interactions resolve, it can hand the player a *recommended path*, not just a verdict. Every uploaded game both trains this core and is a state it can now advise on.

So the honest architecture is **one learned world/value/policy core, queried three ways** — as a fast team-evaluator (distilled value head), as a play-advisor (policy + belief search), and as the coach's explanation layer. Whether tier 1 remains a separate lightweight model or becomes a distilled head of the tier-3 core is an engineering choice, not a conceptual one; both are the same object viewed at different cost.

6.1 Continual learning — the model updates with every upload

"With every game upload the model improves" is **continual (online) learning**, and it has a known failure mode — **catastrophic forgetting**, where new data overwrites old competence. The standard guards are a **replay buffer** (keep training on a mix of old and new games, never new alone), periodic re-fitting rather than pure online updates, and continuing to ingest the *whole* public ladder so the model does not narrow to only the games the current policy likes. Under these guards the flywheel (§8) is a continual-learning loop whose data distribution is steered, deliberately, toward the positions the player actually reaches.

7. Tier 2 — MEDICHAM, the pragmatic middle

Status: built (engine/medicham.js, tests/test-medicham.js). Rain core beats sun core 0.60, mirror 0.51, ~1s / 300 playouts; v1 is sequential-singles (honest scope, true doubles is roadmap).

MEDICHAM sidesteps learning dynamics by *specifying* them: it uses CHOMP's exact damage engine — the same pKO-over-16-rolls threat scoring as tier 1 — as a hand-built T for the dominant effect (damage), pair it with cheap policies (best-damage heuristics, or behaviour-cloned move priors from ABRA's *sets*), and roll a matchup out a few turns with a handful of stochastic samples to average over damage rolls. It is grounded in real math, requires no model training, and reuses code that already exists and is already tested. It is the right first *playout* model precisely because its errors are known and bounded (it ignores multi-turn tactics), and it is a strong baseline

against which any learned tier-3 model must justify its cost.

8. DITTO — team optimisation, the outer loop

Status: built (`engine/ditto.py`). Uses JOLTEON as evaluator against a gauntlet of real ladder teams, double-oracle rounds, usage-weighted threat coverage, and a bias report. Its headline finding is a *negative* one, stated plainly in §12: optimising against JOLTEON Goodharts it (a "90%" team is really ~12% by MEDICHAM), which is exactly why finalists are vetted by Tier 2. Rarity shrinkage (§4.4) and the proven-meta pool are the guards; MEDICHAM is the check.

Given any tier as an evaluator $\hat{E}[\text{win} \mid \text{team}]$, optimise the team. Formally, choosing a team t to maximise expected win rate against the **meta distribution** D (which ABRA measures):

$$t^* = \operatorname{argmax}_{\{t \in \text{legal}\}} E_{\{o \sim D\}} [P_{\text{win}}(t, o)].$$

The legal team space is astronomically large (hundreds of species choose six, times sets), so exhaustive search is out. The tractable families:

- **Evolutionary / local search.** Seed from meta archetypes or a user pool, mutate one slot at a time, keep improvements. Cheap, anytime, and matches how humans iterate.
- **Bayesian optimisation / bandits.** Treat team evaluation as expensive-noisy; use a surrogate over the team-feature space and an acquisition rule to pick the next team to evaluate; **UCB**-style regret bounds apply. Natural when tier-2/3 evaluations are costly.
- **Game-theoretic (the principled version).** The meta is not fixed while you optimise — if you find a team that beats the field, the field adapts. The right object is a **Nash of the team-selection metagame**, approximated by **double-oracle / PSRO** (Policy-Space Response Oracles): alternately compute a best-response team to the current population and add it, growing toward equilibrium. This is exactly the "iterate on itself" the flywheel calls for, and it is the game-theoretic baseline VGC-Bench implements.

Coarse-to-fine ties the tiers together: tier 1 ranks thousands of candidate teams, tier 2 re-ranks the top few hundred, tier 3 vets the handful of finalists. No expensive evaluation is ever spent on an obviously bad team.

9. The self-improving flywheel, formalised

The five ABRA stages form a fixed-point iteration. Let D_k be the meta distribution at round k , M_k the model fit on data up to k , and Π_k the optimised team(s). Then:

```
M_k = Fit(Data_k)           # learn the model from logged
games
Π_k = Optimise(M_k, D_k)    # best team(s) vs the measured
meta
Data_{k+1} = Data_k ∪ Play(Π_k) # play; ingest every opponent
faced
D_{k+1} = Meta(Data_{k+1})   # re-measure the meta
```

This is **expert iteration** — the AlphaZero pattern — with the environment supplied by the live ladder rather than self-play. Its appeal is compounding: each turn of the crank adds data where the current model is *most uncertain* (the games you actually play), which is the most valuable data to acquire. Its dangers are equally real and must be designed against: **feedback loops** (optimising against a meta you also perturb), **distributional collapse** (a model that only ever sees its own preferred games forgets the rest of the ladder — mitigated by continuing to ingest the *whole* public ladder, not only your games), and **non-stationarity** (the meta drifts under you; models must be re-fit, not frozen). The flywheel is powerful *because* it is a control loop, and it must be treated with a control loop's caution.

10. Related work (grounding, 2024-2025)

- **PokéChamp** (Karten, Nguyen, Jin; ICML 2025 spotlight). LLM-augmented **minimax** tree search: the LLM supplies action sampling, opponent modelling, and value estimation inside a two-player game tree; no extra training; Elo 1300-1500 (top 10-30% of humans); ships the largest real-player dataset (3M+ games, 500k+ high-Elo). Directly

validates tier-3-by-search (§5.3).

- **Metamon** (UT Austin RPL; RLC 2025). **Offline RL** with transformers on 5M+ trajectories reconstructed from human Showdown replays plus 20M+ self-play — the offline, learn-from-logs setting that is ABRA's exactly. Validates tier-3-by-learned-policy (§5.4).
- **VGC-Bench** (Angliss et al., 2025). A benchmark for *VGC specifically*, with the combinatorial team-strategy generalisation problem front and centre, and baselines spanning **behaviour cloning, multi-agent RL (PPO), game-theoretic (double-oracle), LLM, and heuristics** — the method menu of §5 and §7. Also extends poke-env with PettingZoo/VGC support.
- **PokéAgent Challenge** (NeurIPS 2025). Establishes Pokémon as a standard testbed for sequential decision-making under uncertainty, with Metamon and PokéChamp as baselines.
- **Classical foundations.** Bradley-Terry (1952) and Elo for paired comparisons (§4.2); CFR / Deep CFR for imperfect-information equilibria (§2); Dreamer and MuZero for learned-model planning (§5); CQL, IQL, Decision Transformer for offline RL (§5.4); PSRO / double-oracle for empirical-game equilibria (§7).

The takeaway: none of the three tiers is speculative. Each corresponds to a demonstrated method in the current literature. What ABRA contributes is not a new algorithm but the **data pipeline and the flywheel** that make these methods self-sustaining on a live, closed-engine format.

11. Evaluation protocol

Every tier is scored on **held-out games** — a temporal split (train on older games, test on newer), never random, to respect meta drift and avoid leakage.

- **Outcome model (tier 1).** Test log-likelihood, **Brier score**, calibration reliability diagram, and AUC. Report against two baselines: the constant 50% predictor, and the Bradley-Terry main-effects-only

model. A model that cannot beat "the higher-usage team wins" is not yet useful.

- **Dynamics model (tier 3).** Per-step next-state log-likelihood and long-horizon rollout divergence (does a 10-turn rollout stay near reality?). Policy quality by **off-policy evaluation** and, ultimately, by **live ladder Elo**, the only unarguable metric.
- **The honest ceiling.** Pokémon has irreducible variance: imperfect information plus damage rolls, accuracy, and speed ties mean even a perfect model cannot predict single games with high accuracy — which is why the strongest published agents plateau around Elo 1300–1500 rather than dominating. A prior internal analysis on a coverage heuristic found game-level correlation ≈ 0 ; that finding stands as the caution. We therefore state success as **calibrated aggregate win-rate improvement of optimised teams over a baseline**, measured over many games, not per-game oracular accuracy.

12. Feasibility — can we actually build this?

Yes, tier by tier, with honesty about each. **As of v1.4, tiers 1 and 2 and the DITTO outer loop are built and measured; tier 3 remains an honest research effort.**

- **Tier 1 (JOLTEON) is built (v2).** A Bradley-Terry logistic model over per-species strengths, now with **rarity-aware L2 shrinkage** (a species is trusted in proportion to its sample size — seen 25×, its rating is pulled toward neutral; seen 1000×, it is trusted) plus speed-edge and firepower-edge features from the dynamics model. On a fresh 4,999-game pull (temporal split, humans only): **~55% held-out accuracy vs ~49% for a coin flip**, Brier ~ 0.25 (calibrated). The number sits 55–57% depending on the exact games/split — modest, exactly as this domain's variance predicts, but genuinely above chance from team composition alone. Honest ablation: the dynamics features *tie* species-only at this scale (firepower earns weight +0.30; speed-edge is noise), and the rarity shrinkage costs no accuracy while removing the overfit

an optimiser could exploit. Antisymmetry and calibration are pinned by `tests/test-jolteon.py`.

- **Tier 2 (MEDICHAM) is built.** A Monte-Carlo rollout over CHOMP's exact damage engine with a heuristic (behaviour-cloned-later) move policy, sampling damage rolls. Sanity holds: rain core beats sun core **0.60**, mirror match **0.51**, ~1s per 300 playouts. v1 is sequential-singles (one active per side) — honest scope; true doubles turns are roadmap. Pinned by `tests/test-medicham.js`.
- **The DITTO outer loop is built** — and its most important output is a *negative* result stated plainly. Optimising a team to maximise JOLTEON's score **Goodharts the evaluator**: DITTO finds rare species JOLTEON overrates and stacks them, producing a team JOLTEON scores at **90%** that MEDICHAM's grounded rollouts reveal is **~12%**. This is caught, not hidden: DITTO now (a) restricts to a proven-meta pool, (b) uses **usage-weighted threat coverage** so it guarantees an answer to high-bring threats (Basculegion) and ignores rare ones (Camerupt), (c) prints a **bias report** showing where rarity shrinkage suppresses a pick, and (d) vets every finalist with MEDICHAM. The JOLTEON-low / MEDICHAM-high disagreement is itself a signal — a genuine sleeper the data has not yet confirmed. This is the tiered design (§3) earning its keep.
- **The dynamics model is built** (`engine/dynamics.js`): the per-turn stream now yields observed speed order (186 species, with Choice-Scarf detection) and observed damage distributions (1,170 attacker|move pairs). This is the empirical grounding tier 3 will calibrate against, and it already feeds tier 1 and KADABRA.
- **Tier 3 (SLOWKING) is a research effort, scaffolded not trained.** A learned dynamics model, belief tracking, and equilibrium search (or offline-RL policy) is the frontier the cited papers occupy. It is *possible* — they prove it — but it needs real compute, careful offline-RL practice, and sustained work, judged against the tier-2 baseline before its cost is accepted. The interface is fixed (`engine/slowking.py`) and the data toward it is already collecting: **30,611 per-turn state**

transitions over 4,999 games, with raw logs archived so nothing must be re-pulled. Grey-box structure (§5.1) and a warm start from open Showdown data are the levers that make it tractable.

The disciplined path — ship tier 1, keep the flywheel turning, add tier 2 for finalist vetting, treat tier 3 as a research track whose bar is set by tier 2 — is now demonstrated end-to-end, not just proposed.

13. References

1. Karten, S., Nguyen, A. L., Jin, C. (2025). *PokéChamp: an Expert-level Minimax Language Agent*. ICML 2025 (spotlight). arXiv:2503.04094.
2. UT Austin RPL (2025). *Metamon: Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers*. RLC 2025. arXiv:2504.04395.
3. Angliss, C. et al. (2025). *VGC-Bench: A Benchmark for Generalizing Across Diverse Team Strategies in Competitive Pokémon*. arXiv:2506.10326.
4. *PokéAgent Challenge*, NeurIPS 2025.
5. Bradley, R. A., Terry, M. E. (1952). *Rank Analysis of Incomplete Block Designs*. Biometrika.
6. Zinkevich, M. et al. (2007). *Regret Minimization in Games with Incomplete Information (CFR)*. NIPS.
7. Kumar, A. et al. (2020). *Conservative Q-Learning for Offline RL*. NeurIPS.
8. Kostrikov, I. et al. (2021). *Offline RL with Implicit Q-Learning (IQL)*.
9. Chen, L. et al. (2021). *Decision Transformer: RL via Sequence Modeling*. NeurIPS.
10. Schrittwieser, J. et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero)*. Nature.
11. Lanctot, M. et al. (2017). *A Unified Game-Theoretic Approach to Multiagent RL (PSRO)*. NeurIPS.

- 12.Chen, S., Joachims, T. (2016). *Modeling Intransitivity in Matchup Outcomes (Blade-Chest)*. ICML.
- 13.Brown, N. et al. (2020). *Combining Deep RL and Search for Imperfect-Information Games (ReBeL)*. NeurIPS.
- 14.Schmid, M. et al. (2021/2023). *Player of Games / Student of Games: unified search + learning across perfect- and imperfect-information games*. Science (2023).
- 15.Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero)*. ICML.
- 16.Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel (Gumbel MuZero)*. ICLR.
- 17.Vinyals, O. et al. (2019). *Grandmaster level in StarCraft II (AlphaStar)*. Nature; and *AlphaStar Unplugged* (2023), large-scale offline RL.
- 18.French, R. (1999). *Catastrophic Forgetting in Connectionist Networks*. Trends in Cognitive Sci.
- 19.Hooper, W. (2026). *CHOMP white paper, validation report, and referee report* — the Gen-9 damage pipeline, the pKO-over-16-rolls threat score, and the coverage-based bring-4, used here as JOLTEON's feature generator and MEDICHAM's dynamics. [Pokemon/CHOMP/docs/](#).

Companion documents. [ABRA white paper](#) · [Executive summary](#) · [Technical documentation](#)

SLOWKING: Offline Belief-State Search for a Closed, Simultaneous-Move Battle Engine

Learning a Tier-3 game solver for Pokémon Champions from logged replays alone

Version 1.0 · 2026-07-23 · Will Hooper · ABRA

*The definitive design for ABRA's deepest model. It states a problem that, in this exact combination, is not solved in the published literature — equilibrium-aware search in a **closed, simultaneous-move, imperfect-information** commercial game engine that we may only observe **offline** — derives the algorithm end to end, grounds every component in the 2020–2025 record (ReBeL, Student of Games, PokéChamp, Metamon, MuZero/Gumbel, CFR, offline RL), and gives an honest, staged feasibility plan built on data ABRA already collects. Living document; updated in the same pass as the code it describes. Companion: [research roadmap](#), [simulator white paper](#).*

Abstract

ABRA collects a growing corpus of Pokémon Champions (Reg M-B) replays — a two-player, zero-sum, *simultaneous-move*, imperfect-information stochastic game whose transition dynamics are implemented by a **closed engine we cannot query**. SLOWKING is the model that plays this game near-optimally: it searches over **belief states** — distributions over what the opponent is hiding — to return an equilibrium-aware policy rather than a greedy best move. The published state of the art for imperfect-information search

(ReBeL, Student of Games, Player of Games) assumes either a *known* simulator or the ability to interact with the environment; the strongest Pokémon agents (PokéChamp, Metamon) either wrap a *reconstructed open-source* engine or learn model-free from logs without equilibrium search. SLOWKING targets the intersection none of them occupy: **equilibrium search over a learned model of a closed engine, trained purely offline, in a simultaneous-move doubles setting**. We contribute (i) a formalisation of Champions as a partially observable stochastic game with a public belief state; (ii) a **grey-box world model** that learns only the *residual* over CHOMP's exact damage calculator, shrinking the learning problem to the parts CHOMP cannot express (status chains, secondary effects, switch dynamics, position); (iii) a belief representation and offline update mined from revealed-set statistics; (iv) a value/policy pair trained offline with conservative RL and warm-started by a behaviour-clone of real ladder play; and (v) a tractable belief-state search that solves each simultaneous-move subgame to a *mixed* strategy and prunes the combinatorial action space with a Gumbel policy-improvement operator. We specify an evaluation protocol centred on **exploitability**, not just win rate, and give a staged build plan whose early stages are startable today on ABRA's data. We do not claim SLOWKING is trained; we claim the problem is well-posed, the design is sound, and the foundation is in place.

1. Introduction

A competitive VGC player answers three nested questions: which six to build (DITTO), which four to bring (CHOMP), and *how to play the resulting game*. The third is the hardest and the highest-ceiling, and it is where every shallow model in ABRA stops. JOLTEON (Tier 1) scores teams in one forward pass but has no notion of a turn. MEDICHAM (Tier 2) plays a matchup out with exact damage and a behaviour-cloned policy, but it is a *fixed-policy rollout*: it never asks "what is my opponent's best response to this line, and what should I mix to be unexploitable?" SLOWKING is the model that asks that question and answers it with search.

Three properties make this genuinely hard, and their *conjunction* is what places SLOWKING outside the existing literature:

1. **The engine is closed.** Champions is not the Smogon format served by the open-source `pokemon-showdown` simulator that `poke-env` exposes to RL agents. Its ruleset and stat system are not a queryable environment. We cannot roll the true dynamics forward for a hypothetical state, so we must *learn* a model of them from observation — the offline model-learning / system-ID setting.
2. **Moves are simultaneous.** Both players commit each turn without seeing the other's choice. The per-turn subgame is therefore a matrix game whose optimal solution is generally a *mixed* Nash equilibrium. A searcher that assumes a deterministic opponent is exploitable exactly at the decision points that matter.
3. **Information is imperfect, and we are offline.** We see the opponent's six at preview and each set only as it is revealed. The relevant object is a belief over hidden information, and we must learn to track and search over it *without ever interacting with the real engine* — only from logged trajectories, with all the distribution-shift hazards offline RL is heir to.

The rest of this paper formalises the game (§2), explains precisely why the standard playbook does not port unmodified (§3), derives SLOWKING component by component (§4), specifies offline training (§5) and the tractability machinery (§6), gives an honest evaluation protocol (§7) and feasibility assessment (§8), situates the work against prior art (§9), and states what is genuinely novel (§10).

2. The game, formally

A Champions battle is a **partially observable stochastic game (POSG)**: a finite set of players $i \in \{1, 2\}$, a state space S , per-player action sets, a stochastic transition kernel, a zero-sum terminal reward, and per-player observations.

- **State** $s \in S$ comprises both teams' species, full sets (moves, item, ability, the Champions SP spread), current HP, status, stat stages, field conditions (weather, terrain, screens, Trick Room, Tailwind timers), and which Pokémon are active. Level 50; no Tera.
- **Actions.** At each turn player i chooses *simultaneously* from $A_i(s)$: for each of two active Pokémon, a move (with target) or a switch. The joint action is $a = (a_1, a_2)$. Team preview is a special first move: an ordered bring of four from six, $|A_i| = 6 \cdot 5 \cdot 4 \cdot 3 = 360$.
- **Transition** $T(s' | s, a)$ is stochastic — damage rolls (16 outcomes, 85–100%), accuracy, secondary-effect procs, critical hits, speed ties. **This kernel is the closed engine; reproducing it is the crux.**
- **Reward** is terminal and zero-sum: +1 win, -1 loss.
- **Observation** $o_i = O_i(s)$: own team fully; of the opponent, the six species at preview and each move/item/ability only once used or triggered.

Because information is imperfect, the decision-relevant object is not the world state s but the **information state** — the history of one player's own actions and observations — and, for search, the **public belief state (PBS)**: the common-knowledge distribution over the pair of information states, given the public history. Following ReBeL, converting the game to a game *over public belief states* makes an imperfect-information game behave, for the purposes of value-function search, like a perfect-information one: the value of a PBS is well-defined and learnable, and search can proceed on PBS nodes. This PBS reformulation is the theoretical backbone of §4.4.

Two structural facts govern everything downstream: **simultaneous moves** $\Rightarrow$ **mixed strategies** (each turn is a matrix game; the solution is a distribution), and **imperfect information** $\Rightarrow$ **belief states** (we plan over b , not s). The team-building layer above the battle is a **Bayesian game** — commit a team before seeing the opponent's — and is DITTO's problem, evaluated by the battle values SLOWKING defines.

3. Why the standard playbook does not port unmodified

The modern recipe for superhuman imperfect-information play — ReBeL and Student of Games — is **self-play RL + search over public belief states, with a learned value/policy and a CFR-style subgame solver**. It is proven to converge toward Nash in two-player zero-sum games and reaches superhuman poker and strong Scotland Yard / chess / Go. We adopt its skeleton. But three assumptions it leans on are violated here, and each violation forces a design change:

- **It assumes a known, queryable simulator (or environment access) for self-play.** ReBeL and SoG generate their own trajectories by playing the game forward through a known ruleset. We *cannot*: the Champions engine is closed. **Consequence:** we must first *learn* the transition model T_θ (§4.1) and do all training offline (§5). This is the single biggest departure and the reason SLOWKING is a research effort rather than a re-implementation.
- **Canonical treatments are turn-based (sequential).** Poker and Scotland Yard reveal actions in order; the PBS machinery is usually derived for sequential moves. Champions is *simultaneous*. **Consequence:** the subgame at each node is a genuine matrix game, and the solver must return a mixed strategy over the joint action (§4.5), not a sequence of single-agent decisions.
- **Offline RL is fragile.** Learning a value/policy purely from logged data invites distribution-shift and value over-estimation on actions the data never took. **Consequence:** we use *conservative* offline RL (CQL/IQL-style pessimism) and anchor the policy to a **behaviour-clone** of real play so search never wanders far from supported states (§4.3, §6).

PokéChamp and Metamon confirm the domain is tractable but sit on the other side of these constraints: PokéChamp runs a minimax LLM agent over the *open-source* engine with no equilibrium guarantee and no learned dynamics; Metamon trains strong *model-free* offline-RL agents from 5M+ reconstructed Showdown trajectories but does no belief-state search.

SLOWKING = the belief-state searcher, over a *learned* model of a *closed* engine, in the *simultaneous-move* setting — the box none of them tick.

4. The SLOWKING architecture

SLOWKING has five components: a grey-box world model, a belief tracker, an offline value/policy pair, a belief-state search that ties them together, and a per-turn mixed-strategy subgame solver.

4.1 Grey-box dynamics $T\theta(s' | s, a)$ — learn only the residual over CHOMP

The naïve approach learns the entire transition kernel from scratch. That is wasteful: we already own an *exact* partial simulator of the dominant effect. CHOMP's `champ-model.js` computes, deterministically and correctly, the damage distribution of any move in any matchup — STAB, type chart, weather, items, abilities, spread, the full 16-roll range — validated against the reference calculator. So we factor the kernel:

$$T(s' | s, a) = T_{\text{known}}(s' | s, a; \text{CHOMP}) \otimes T\theta_{\text{resid}}(s' | s, a)$$

T_{known} applies CHOMP's exact damage and the mechanically certain effects (fainting, weather setting, speed order under known modifiers). $T\theta_{\text{resid}}$ — the only learned part — models what CHOMP does not express: status infliction and its downstream (sleep turns, burn/paralysis dynamics), secondary-effect probabilities, switch decisions' consequences, redirection and position in doubles, and the residual between predicted and observed damage. This is **grey-box system identification**, and it shrinks the learning target to a small, well-supported delta. Two properties of ABRA's data make it learnable now: we store the **per-turn stream** (move order → observed speed, exact damage % per move) for 30k+ turns, and the **behaviour-clone** (`data/move-priors.json`) already captures the empirical action policy the dynamics must be consistent with. The model is calibrated against `data/dynamics.json`'s observed per-move damage distributions — MEDICHAM's grounding becomes SLOWKING's supervision. *Literature:* MuZero and Dreamer show learned models suffice for planning; Metamon

shows Showdown dynamics are learnable at scale from logs.

4.2 Belief representation and offline update

The belief $\mathbf{b}$ is a distribution over the opponent's hidden variables: which four of their six were brought, and each brought Pokémon's full set (moves/item/ability/spread) consistent with what has been revealed. We represent $\mathbf{b}$ as a **factored particle set**: a bag of concrete opponent configurations, each weighted, sampled from priors and reweighted by observations. The priors are *learned from the corpus, not assumed*: ABRA's usage model gives bring/lead rates; the behaviour-clone gives per-species set and move priors; revealed moves/items hard-constrain the support (a Pokémon that has clicked Protect and Moonblast cannot be a four-attacks set). The update is Bayesian filtering: on each observation $\mathbf{o}$, reweight each particle by $P(\mathbf{o} \mid \text{particle})$ under $T\theta$, resample, and prune particles the observation has falsified. Because every prior is mined from stored data, belief tracking is itself an *offline-trained* component — no environment access required.

4.3 Value and policy from offline data

Search needs a **value function** $V(\mathbf{b})$ (the equilibrium value of a public belief state) and a **policy prior** $\pi(\mathbf{a} \mid \mathbf{b})$ (a distribution to guide and prune search). We obtain both offline:

- **Policy prior** is initialised from the behaviour-clone — the empirical action distribution of real ladder players, per species and turn context (`policy.js`), already built. This is a strong, data-supported prior that keeps search inside the manifold of plays the data covers, directly mitigating offline distribution shift.
- **Value** is trained with **conservative offline RL** (CQL/IQL-style): learn V from logged returns while penalising optimism on out-of-distribution actions, so the searcher is not seduced by states the data never validated. The value target is bootstrapped through the learned model and regressed toward realised game outcomes — the reward is terminal and known (win/loss), which is a gift for credit assignment.

Both are refined by search via a policy-improvement loop (§6): the searched policy at a state becomes a supervised target for π , and the searched value becomes a target for V — the ReBeL/SoG self-improvement loop, run **offline through the learned model** instead of the real engine.

4.4 Belief-state search

At decision time SLOWKING builds a search tree whose nodes are **public belief states**. From the current PBS it expands a few plies: at each node it (a) samples opponent configurations from the belief (§4.2), (b) proposes candidate joint actions from the policy prior (§4.3), (c) rolls them through the grey-box model T_θ (§4.1) to child PBSs, and (d) backs up values from the learned V at the leaves. The novelty over perfect-information MCTS is that expansion and back-up operate on *belief* nodes and that each internal node is solved as a *simultaneous-move subgame* (§4.5) rather than a max/min. This is exactly the ReBeL construction — value-function search over PBS with a CFR-style equilibrium solver at subgames — adapted to (i) a learned model in place of a known one and (ii) simultaneous rather than sequential moves. *Literature*: ReBeL (PBS value search + CFR), Student of Games / Player of Games (growing-tree CFR with learned value, sound convergence).

4.5 The simultaneous-move subgame solver

Each turn is a matrix game: rows are player 1's legal joint actions (per-slot move/switch), columns are player 2's, and the payoff of a cell is the searched value of the resulting child PBS. Its solution is a **mixed Nash equilibrium** — a distribution over actions for each side. We compute it with **regret-matching / CFR** iterated to low exploitability inside the subgame, yielding $\sigma_i(a_i | b)$. SLOWKING then plays its own mixed strategy σ_1 , which is unexploitable *within the searched horizon* by construction. Two practical points: the combinatorial action space (two slots, moves $\times$ targets $\times$ switches) is pruned to the top candidates by the policy prior before the matrix is formed (§6), and the subgame payoffs reuse the same T_θ/V as the outer search, so the solver adds equilibrium reasoning without a second model.

5. Offline training pipeline

Every component trains from the durable store, never from the live engine, and improves as the store grows — the flywheel, applied to Tier 3:

1. **Trajectories.** Convert each stored replay into `(s, o, a, r)` transitions via a game-spec encoder (`engine/game-spec.js`, the roadmap's Paper 1 — built as the first concrete step; see §8). Champions' per-turn stream + raw-log archive means this is a re-parse, never a re-pull.
2. **Grey-box residual T_0** is fit to observed transitions with CHOMP's `T_known` subtracted out, supervised by observed damage (`dynamics.json`) and move order.
3. **Belief priors** are the existing usage model + behaviour-clone; no training beyond what ABRA already computes.
4. **Value/policy** are trained with conservative offline RL, warm-started by the behaviour-clone, then refined by searched targets (§4.3).
5. **Search-and-distil** closes the loop: run belief-state search over T_0 on sampled states, distil the searched policy/value back into π/V , repeat. This is self-improvement without self-play against a real engine — the offline analogue of ReBeL's training loop.

6. Making it tractable

Three levers keep the cost bounded. **(a) Policy-guided pruning:** the behaviour-cloned prior collapses each turn's enormous action set to a handful of plausible candidates before any search — you do not simulate every path; the policy proposes the few viable ones (the AlphaStar/Gumbel insight). **(b) Gumbel policy improvement:** at the root we select actions by sampling-without-replacement with the Gumbel-Top-k trick, which guarantees a policy-improvement step even with *very few* simulations — Gumbel MuZero learns reliably at as few as two simulations per move, the property that makes second-scale planning feasible. **(c) Coarse-to-fine across tiers:** SLOWKING is only ever invoked on the handful of states that matter — JOLTEON screens thousands of teams, MEDICHAM re-ranks finalists, and SLOWKING vets the

last few or the critical in-game turns. No expensive belief search is spent where a cheap model suffices.

7. Evaluation protocol

Win rate alone is misleading in a high-variance game, and a searcher can win yet be exploitable. We evaluate on four axes:

1. **Head-to-head** vs the ladder of baselines already in ABRA — the behaviour-clone policy, MEDICHAM's fixed-policy rollout, and a greedy-damage agent — under identical team samples, reporting win rate with confidence intervals.
2. **Exploitability proxy.** Against a best-response opponent computed within the learned model, measure how much the opponent can gain — the operational definition of "how close to equilibrium." A model-free agent has no such guarantee; this is where belief search should distinguish itself.
3. **Calibration.** Brier score and reliability curves on SLOWKING's win-probability estimates vs realised outcomes on held-out games (temporal split), as we already do for JOLTEON.
4. **Ablations.** Grey-box vs learn-from-scratch dynamics; behaviour-clone prior vs uniform; mixed-Nash subgame solver vs deterministic max — each isolates a claimed contribution.

Crucially, results are reported against the domain's irreducible variance (§ the simulator paper's honesty about a ~55–57% team-only ceiling): SLOWKING's job is to raise the *in-game* decision quality above MEDICHAM's fixed policy and to be *unexploitable*, not to manufacture certainty the game does not contain.

8. Feasibility, staged and honest

We do not claim SLOWKING is built. We claim it is well-posed and that the on-ramp exists:

- **Ready today.** The offline corpus (30k+ turn transitions, growing hourly), the exact damage prior (CHOMP), the observed-dynamics supervision (`dynamics.json`), and the behaviour-clone policy prior (`move-priors.json`) are all built and tested. Two of the five roadmap papers (the game-spec/data and the value/policy) are startable now on this data.
- **First concrete step (this release):** `engine/game-spec.js` — the state/observation/action encoding and a converter that turns stored replays into `(s, o, a, r)` trajectories. This is Paper 1's deliverable and it makes SLOWKING a build with a first commit, not only a design.
- **The research.** The grey-box residual model, the belief-state search, and the offline search-and-distil loop are genuine research — the frontier ReBeL/SoG/Metamon occupy. They need real compute, careful offline-RL practice, and sustained work, and each should be judged against the MEDICHAM baseline before its cost is accepted. Recent results prove this class of method *works*; none prove it is cheap.

The disciplined path: ship the data encoder and value/policy on current data; stand up the grey-box model against observed dynamics; add belief-state search last, gated by exploitability against MEDICHAM. Every stage is useful on its own, and every stage feeds the models above it.

9. Relationship to prior work

- **ReBeL** (Brown et al., NeurIPS 2020) — RL+search over public belief states with a CFR subgame solver; converges toward Nash in 2p0s games; superhuman poker. SLOWKING's search skeleton, adapted to a learned model and simultaneous moves.
- **Student of Games** (Schmid et al., Science Advances 2023) and **Player of Games** (Schmid et al., 2021) — a unified sound algorithm across perfect/imperfect information via growing-tree CFR with learned value; strong in chess, Go, poker, Scotland Yard. The template for combining search, learning, and game-theoretic reasoning.
- **MuZero** (Schrittwieser et al., Nature 2020) and **Gumbel MuZero** /

Policy improvement by planning with Gumbel (Danihelka et al., ICLR 2022) — planning with a learned model; policy improvement with few simulations. SLOWKING's learned-model and low-simulation planning basis.

- **PokéChamp** (Karten et al., ICML 2025) — expert-level minimax *LLM* agent over the open-source engine (~1500 Elo, 1M+ game dataset), no learned dynamics, no equilibrium guarantee. A domain proof and a baseline, on the other side of the closed-engine and equilibrium constraints.
- **Metamon** (UT Austin RPL, RLC 2025) — human-level *model-free* offline RL from 5M+ reconstructed Showdown trajectories. Proves offline learning of Showdown play at scale; no belief search.
- **VGC-Bench** (Angliss et al., 2025) — a doubles-VGC benchmark with PSRO/self-play baselines; the double-oracle machinery DITTO's outer loop mirrors.
- **CFR** (Zinkevich et al., 2007) and **offline RL** — CQL (Kumar et al., 2020), IQL (Kostrikov et al., 2021), Decision Transformer (Chen et al., 2021) — the equilibrium and offline-learning foundations.

10. What is genuinely new

Each ingredient exists; their *combination*, in this domain, does not appear in the literature:

Equilibrium-aware belief-state search over a learned grey-box model of a closed battle engine, trained entirely offline from public replays, warm-started by a behaviour-clone mined from the same corpus, solving simultaneous-move subgames to mixed strategies, in doubles VGC.

ReBeL/SoG assume a known model or environment access; PokéChamp uses an open engine and no equilibrium search; Metamon is model-free with no search; MuZero/Gumbel are perfect-information or single-agent. The grey-box factorisation that learns only the residual over an exact damage

calculator (CHOMP) is, to our knowledge, a new device for making offline model-learning tractable in this setting, and it is only possible because ABRA already owns both the exact partial simulator and the observed-dynamics supervision to fit the residual against. That is the paper's thesis: **the closed engine is not a wall but a residual, and the corpus we already store is enough to learn it.**

11. References

1. Brown, N., Bakhtin, A., Lerer, A., Gong, Q. (2020). *Combining Deep Reinforcement Learning and Search for Imperfect-Information Games (ReBeL)*. NeurIPS. arXiv:2007.13544.
2. Schmid, M. et al. (2023). *Student of Games: A unified learning algorithm for both perfect and imperfect information games*. Science Advances 9(46). DOI:10.1126/sciadv.adg3256.
3. Schmid, M. et al. (2021). *Player of Games*. arXiv:2112.03178.
4. Schrittwieser, J. et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero)*. Nature 588.
5. Danihelka, I., Guez, A., Schrittwieser, J., Silver, D. (2022). *Policy improvement by planning with Gumbel*. ICLR.
6. Karten, S., Nguyen, A. L., Jin, C. (2025). *PokéChamp: an Expert-level Minimax Language Agent for Competitive Pokémon*. ICML (spotlight). arXiv:2503.04094.
7. UT Austin RPL (2025). *Metamon: Human-Level Competitive Pokémon via Scalable Offline RL with Transformers*. RLC. arXiv:2504.04395.
8. Angliss, C. et al. (2025). *VGC-Bench: Generalizing Across Diverse Team Strategies in Competitive Pokémon*. arXiv:2506.10326.
9. Zinkevich, M., Johanson, M., Bowling, M., Piccione, C. (2007). *Regret Minimization in Games with Incomplete Information (CFR)*. NIPS.
10. Kumar, A., Zhou, A., Tucker, G., Levine, S. (2020). *Conservative Q-Learning for Offline RL*. NeurIPS.
11. Kostrikov, I., Nair, A., Levine, S. (2021). *Offline RL with Implicit Q-*

Learning (IQL).

12.Chen, L. et al. (2021). *Decision Transformer: RL via Sequence Modeling*. NeurIPS.

13.Hafner, D. et al. (2023). *Mastering Diverse Domains through World Models (DreamerV3)*. arXiv:2301.04104.

Companion documents. [Research roadmap](#) · [Simulator white paper](#) · [Cheat sheet](#) · [Executive summary](#)

SLOWKING — Research Roadmap

Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver

Version 0.1 · 2026-07-23 · Will Hooper · ABRA Tier-3

*These are not decorations. Each tag in Slowking's Library is a **paper I will write and a component I will build** on the road to a real Tier-3 solver. This document turns the five research pillars into an ordered plan: what each paper proves, what it produces in code, and how it plugs into the pipeline. SLOWKING is the hard one — a genuine research effort — and this is how it gets built, honestly, one pillar at a time. The math foundation is already laid in [the simulator white paper](#); this is the execution plan on top of it.*

The target

Champions is a two-player, zero-sum, **simultaneous-move, imperfect-information** stochastic game. Solving it well means: track what the opponent is hiding (a belief), look ahead over how the game could branch, and play the move that is best against their best response — a **mixed-strategy equilibrium**, not a single "best move." SLOWKING is the model that does that. No single paper gets us there; five do, in order.

The five papers (each a real deliverable)

Paper 1 — POSG / Markov game: the formalism and the offline dataset

Proves: that a Champions battle is a partially-observable stochastic game, and that our logged replays are a valid offline dataset of trajectories (s, o, a, r) to learn from. **Produces:** `engine/game-spec.js` — a precise state/observation/action encoding, and a converter that turns every stored replay (we already keep the per-turn stream + raw logs) into training trajectories. **Status:** the data exists (30k+ turn transitions); the encoding is the first build.

Paper 2 — Learned dynamics / world model: $T\theta(s' | s, a)$

Proves: we can learn the transition kernel of the closed Champions engine from logged transitions alone, grey-boxed with CHOMP's exact damage as a prior so the model only has to learn what CHOMP doesn't (status chains, secondary procs, switch dynamics, position). **Produces:** a trained forward model that, given a state and a joint action, predicts the next state distribution — the thing we cannot query from the real engine. Calibrated against the observed damage distributions (`data/dynamics.json`) and observed move order. **Literature:** MuZero (learned model + planning), Dreamer (latent world models), Metamon (reconstructing 5M+ Showdown trajectories offline).

Paper 3 — Mixed Nash under simultaneous moves: the per-turn subgame solver

Proves: each turn is a matrix game whose solution is a mixed equilibrium; we can approximate it tractably at VGC branching factors. **Produces:** a subgame solver (regret-matching / CFR-style) that outputs a *distribution* over actions, so SLOWKING plays unexploitably instead of deterministically — the thing that makes it robust to a thinking opponent. **Literature:** CFR (Zinkevich et al.), and its modern deep variants.

Paper 4 — Belief-state search: planning over what the opponent hides

Proves: searching over belief states (distributions consistent with what's been revealed), not raw states, is both necessary (imperfect info) and feasible with the learned model. **Produces:** the search itself — a few plies deep, maintaining and updating the opponent belief from each observation, combining Papers 2 and 3. This is the core of `engine/slowking.py`'s `belief_search`, currently a stub. **Literature:** ReBeL, Player-of-Games, Student-of-Games (belief-state search that reaches superhuman play in imperfect-information games).

Paper 5 — Offline RL value/policy: making it strong without a live engine

Proves: we can train a value head and policy from logged data without ever interacting with the real engine, avoiding the distributional-shift traps that sink naive offline RL. **Produces:** the value function `V(belief)` and a policy prior that guides the search (so we prune to the few viable lines instead of simulating every path) — the piece that makes SLOWKING fast enough to be useful. **Literature:** CQL, IQL, Decision Transformer; AlphaStar / Gumbel-MuZero for policy-guided pruning.

How they compose

```
Paper 1 (spec + data) -> Paper 2 (world model Tθ) -> Paper 4 (belief search)
                                     ^               ^
                                     Paper 5 (value/policy) -----/
                                           Paper 3 (mixed-Nash subgame solver)
feeds the search's per-turn choice
```

SLOWKING = Paper 2's model + Paper 5's value/policy, searched by Paper 4 over beliefs, with Paper 3 solving each turn's subgame. Papers 1 and 5 are the near-term, data-ready builds; 2-4 are the research.

How SLOWKING pays off the moment it exists

- **KADABRA** gets the deep line — "the equilibrium-best play here was X" — with no interface change.
- **DITTO** gains a Tier-3 finalist vetter: JOLTEON screens thousands, MEDICHAM re-ranks finalists, SLOWKING vets the last few. (Coarse-to-fine — being wired now with MEDICHAM as the middle tier.)
- **MEDICHAM** rollouts get a smarter default policy (Paper 5's policy prior) than the behaviour-clone.

Honest status

Zero of the five are trained today; the interface (`engine/slowking.py`) and the data pipeline that feeds them are built and the corpus is growing hourly. Papers 1 and 5 are startable now on current data. This is a multi-month research track, stated plainly — not a claim that it is done.

Related. [Simulator white paper](#) · [Cheat sheet](#) · [Executive summary](#)

KADABRA — Coach Product Spec

Key Analysis of Decisions, Advice & Better Replay Annotation

Version 0.1 (spec) · 2026-07-22 · Will Hooper

*The interactive coaching front-end of the ABRA platform. ABRA collects and models the ladder; **KADABRA is the psychic that shows it back to you** — it walks you through your own game turn by turn, recreates each scene, tells you the better play, and answers your questions before you move on. Abra → Kadabra: the data platform's teaching evolution.*

1. What it is

A **chatbot coach** for a single Champions replay. You paste a replay link; KADABRA reviews the game *with* you, one decision at a time. It is not a wall-of-text report (that is the `coach.js` summary). It is a conversation that stops at each meaningful turn, **reconstructs the scene**, evaluates your play against the models, and waits for your questions before advancing.

It replaces "send Claude a link and have it watch the game" with a purpose-built, instant, repeatable tool — and every game you review is silently added to ABRA's database and folded into the model (the flywheel, from the coaching seat).

2. The core loop (per turn)

1. **Reconstruct the scene.** From the parsed log, KADABRA rebuilds the board at this turn: the two active Pokémon per side, HP bars, status, stat stages, weather/terrain/screens, and what is known of each team. It *shows* this, not just describes it — a rendered board state, so you see

the scenario the way it looked in-game.

2. **State what happened.** "You led Pelipper + Swampert into Gengar + Hydrapple; rain is up; Gengar mega'd and put Swampert to sleep."
3. **Evaluate the decision.** KADABRA compares the move you made against the model's recommended play (see §4), with the reason: the damage math (CHOMP), the threat read (ABRA usage), and — when available — the deeper line (SLOWKING).
4. **Answer your questions.** You can ask anything — "why not Protect?", "would Ice Beam have KO'd?", "what if they didn't switch?" — and KADABRA answers from the same models, running the damage calc or a rollout on demand.
5. **Advance on your say-so.** Only when you're ready does it move to the next decision point. You control the pace; it never dumps the whole game at once.

3. Scene reconstruction

The parser (`engine/durable-ingest.js` / `engine/coach.js`) already reconstructs the full state progression from the protocol log — switches, moves, damage, fainted, field changes. KADABRA renders that state at any turn as a simple board: two active slots per side with sprite, HP %, status chip, and a field banner (weather/terrain/Trick Room/Tailwind). Because the whole trajectory is reconstructable, the player can also scrub — "show me turn 5" — and KADABRA redraws that scene.

4. The advice — where the models plug in

"The correct play might have been..." is answered at the fidelity available:

- **Now (v1).** CHOMP's exact damage engine gives the KO math for every option (pKO across 16 rolls, speed order), and ABRA's usage model gives the opponent read (their likely set/lead). Together these already justify most coaching calls — "Wave Crash OHKOs their Garchomp in rain; you clicked Protect and gave them a free turn."

- **Later (roadmap). SLOWKING** (the learned belief-search solver, simulator white paper §5) supplies the deep line — the equilibrium-aware best move accounting for what the opponent might do — and **MEDICHAM** rollouts answer "what if" branches. KADABRA's advice quality rises as those models come online, with no change to the interface.

KADABRA is a **consumer** of the models (MEDICHAM, JOLTEON, SLOWKING) and of CHOMP, not a model itself.

5. The background job — the flywheel from the coaching seat

Every replay KADABRA reviews is passed through the ABRA extractor and appended to `data/games.ladder.jsonl` (dedup by id). So a coaching session is also a data-collection event: the opponent's team, sets revealed, and result enter the store and improve the models — exactly the feed-back stage of the flywheel, triggered by the act of reviewing your own game.

6. Interface

A single conversational panel: the reconstructed scene on top, the coach's message below, a text box for your questions, and a "next decision →" control. Optionally a side rail with the turn timeline so you can jump around. Web-based, so it can live alongside the ABRA dashboard.

7. Honest status and scope

- **Buildable now:** the parser, scene reconstruction, CHOMP-based advice, the Q&A loop over damage math, and the background ingest. This is a real v1.
- **Roadmap:** the *depth* of "the correct play" scales with SLOWKING and JOLTEON; until then advice is grounded in exact damage + usage, which is honest and already strong, but single-ply.

- **Own project?** KADABRA is a distinct product (its own app and, when built, its own repository and full doc set), connected to ABRA (data/models) and CHOMP (damage). It is specced here inside ABRA because it is the platform's front-end; it graduates to its own project folder when we build it.

8. The named cast (so the pipeline is legible)

Name	Pokémon	Role
ABRA	Abra	collects live ladder games, stores + models them
JOLTEON	Jolteon (fastest)	instant team-vs-team win probability
MEDICHAM	Medicham (medium)	quick physics rollouts
SLOWKING	Slowking (slow, wise)	deep learned belief-search game solver
DITTO	Ditto	team optimiser vs the meta
CHOMP	Garchomp	the bring-4 / lead-2 engine (exact damage)
KADABRA	Kadabra	the coach — reviews your games and teaches

Related. [ABRA white paper](#) · [Simulator white paper](#) · [Executive summary](#)

ABRA — Architecture Notes

Version 1.0 · Last updated 2026-07-23 · Will Hooper

The design decisions behind ABRA that aren't obvious from any single file: why the models are split across two languages, how the local app runs the real engines on your own machine, and how recency weighting keeps every model tracking the live metagame. Living document; updated with the code.

1. Why two languages (Python and JavaScript)

ABRA's engines are deliberately split. It is not an accident or inconsistency — each half of the system lives in the language that makes it shortest and safest to write.

The rule

- **JavaScript (Node) for anything that touches the battle engine.** CHOMP's damage calculator (`CHOMP/engine/champ-model.js`) is the validated, exact model of Champions damage — types, STAB, weather, items, abilities, the full 16-roll distribution. It is written in JS. Every model that needs *real game mechanics* reuses it directly instead of re-implementing (and re-validating) the damage formula:
- **MEDICHAM** (`engine/medicham.js`) — rollouts call CHOMP's `bestDamage` per hit.
- **KADABRA** (`engine/kadabra.js`) — the coach reconstructs scenes and cross-checks damage.
- **the set-builder, behaviour-clone, dynamics** — all Node, all sharing the same move/typing data. Re-reporting the damage engine to Python

would mean maintaining two copies of the most correctness- critical code in the project. That is the thing we most want to avoid.

- **Python (with numpy) for the statistical / machine-learning core.**

JOLTEON's training (`engine/jolteon.py`) and DITTO's search (`engine/ditto.py`) are linear algebra: logistic-regression gradient descent, vectorised scoring of thousands of teams, rarity-aware regularisation, recency weighting. numpy expresses this in a few lines and runs it fast. Writing weighted logistic regression by hand in JS would be longer and slower with no upside.

The boundary, and why it barely matters to a user

The split falls on a clean seam: **mechanics = JS, learned numeric models = Python**. The one place they meet is DITTO, which uses JOLTEON (Python) as its fast evaluator and MEDICHAM (JS) as its grounded vetter — it shells out to both.

Crucially, **using the models needs almost no Python**: - JOLTEON's *predictions* are a tiny logistic dot-product. We ship the trained weights as JSON and recompute the probability in pure JavaScript in the browser (`web/index.html`) and could do so in Node. You never need Python to *use* JOLTEON — only to *retrain* it. - MEDICHAM, KADABRA, CHOMP are Node end to end. - Only **DITTO** currently requires Python at run time (numpy for the team search). Because that is the sole run-time Python dependency, the planned **Node port of DITTO** removes Python from the live app entirely: JOLTEON weights are already portable, MEDICHAM is already JS, so a JS DITTO closes the loop and the whole `localhost` app runs on Node alone. Retraining JOLTEON stays in Python, offline, where numpy earns its keep.

The Windows Python gotcha (documented because it will bite)

Windows ships a fake `python` command that opens the Microsoft Store instead of running Python. So on a machine without a real Python install, `python engine/ditto.py` prints *"Python was not found; run without arguments to install from the Microsoft Store."* The server (`server.js`) handles this: it

probes `py`, then `python`, then `python3`, skips the Store stub, checks that `numpy` imports, and — if no real Python is found — returns a clear message telling you JOLTEON/MEDICHAM/KADABRA still work and how to install Python for DITTO (`python.org`, "Add to PATH", then `pip install numpy`). This probe is **lazy** (run on the first DITTO/JOLTEON request, not at startup) so the server always starts listening instantly.

2. The local app — running the real engines on your machine

`web/index.html` (mirrored to `app/index.html`, which the server prefers) is the ABRA WORLD site. Opened as a file, it runs **JOLTEON** live in the browser (embedded weights) — that alone needs no server. To run the *other* engines, `start.bat` launches `server.js`, a dependency-free Node HTTP server that: - serves the site from `app/` (falling back to `web/`), - exposes `/api/medicham`, `/api/kadabra` (Node engines), `/api/jolteon`, `/api/ditto` (Python engines), and `/api/stats` (live counts for the town), - runs every computation **on your own machine** — nothing is sent anywhere.

`start.bat` checks Node is installed, opens the browser *after* a short delay (so the first load doesn't race the server), and keeps a window open as the running server. Everything is local; "server" here just means "a small program using your computer's own CPU," not a remote host.

3. Recency weighting — tracking a metagame that moves

The problem: concept drift

A competitive metagame is **non-stationary**. The teams that win shift week to week as players adapt, new cores emerge, and old ones fall off. A model trained with every game counted equally averages over that history, so it is always partly describing a metagame that no longer exists. In machine-learning terms this is **concept drift**, and the standard remedy is to weight

recent data more heavily.

The mechanism: exponential decay with a half-life

Every training game gets a weight that decays exponentially with its age:

$$w_i = 0.5^{(age_days_i / \tau)}$$

where `age_days_i` is how old game i is relative to the newest game in the set, and τ (**tau**) is the **half-life in days**. Interpretation: - a game from *today* has weight ~ 1 , - a game **one half-life** old counts **half**, - **two half-lives** $\rightarrow$ a quarter, **three** $\rightarrow$ an eighth, and so on.

Weights are normalised to mean 1 so the regularisation scale (the rarity-aware L2) is unchanged; only the *relative* influence of recent vs. old games shifts. Setting $\tau \rightarrow \infty$ recovers the old equal-weight behaviour exactly, so decay is a strict generalisation, not a different model.

Where it plugs in

The weights enter the **weighted logistic-regression gradient** in JOLTEON's `fit()`:

$$\text{grad} = X^T (w \odot (p - y)) / \sum w + L2 \cdot \theta / \sum w$$

The same $w_i = 0.5^{(age/\tau)}$ applies identically to the **usage model** (weighted species counts) and the **behaviour-clone** (weighted move frequencies), so JOLTEON, DITTO's meta gauntlet, CHOMP's usage intel, and MEDICHAM's move priors all track the *current* ladder from one shared decay rule. The default half-life is **30 days** (`HALF_LIFE` env var); shorter τ reacts faster to a shifting meta but uses effectively less data, longer τ is steadier but laggier — a bias/variance knob.

Honest status on current data

ABRA's store currently spans ~ 2 days, so at $\tau = 30$ days every game's weight is ~ 1 and decay is a **no-op** — reported honestly (the equal-weight and recency-decayed rows are identical). The mechanism is unit-tested on synthetic 90-day data (oldest game weight 0.33, newest 2.14 at $\tau = 30$ d) and

will engage automatically as the corpus ages past a few weeks. It costs nothing now and means the models never need manual pruning of stale games later — the influence just fades on its own.

Why not simply delete old games?

Because rare-but-still-relevant information (an off-meta answer that occasionally matters) survives in old games, and a hard cutoff throws it away discontinuously. Exponential decay is a soft, continuous down-weighting: old games keep a small vote, recent games dominate, and nothing is ever lost from the store (which stays the durable, replayable record).

Related. [Simulator white paper](#) · [SLOWKING white paper](#) · [Cheat sheet](#) · [Changelog](#)